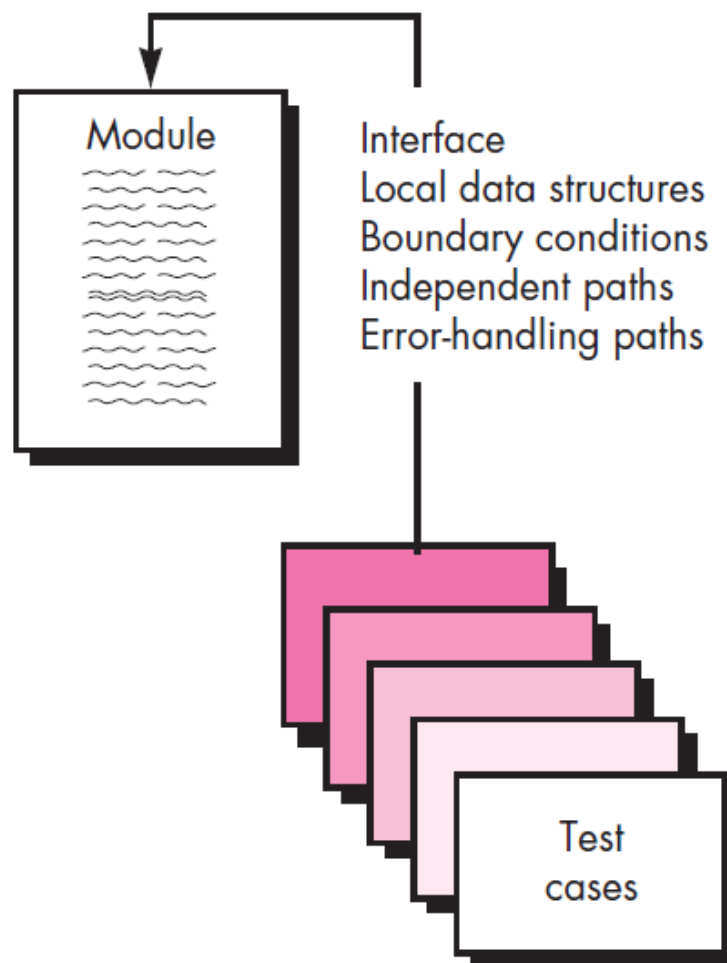


Unit Testing

1 Introduction to Unit Testing

Unit testing focuses verification effort on the smallest unit of software design—typically a software component, module, class, or function.

Unit testing verifies that each individual unit performs its intended function correctly before it is integrated with other units.



Unit testing concentrates on **internal logic and data structures** within a component, not on system-level behavior.

This form of testing is usually conducted by the **software developer** and can be performed in parallel for multiple components.

2 Objectives of Unit Testing

The primary goals of unit testing are:

- To uncover errors within the boundaries of a component
- To verify correct implementation of design logic
- To detect faults early in the development process
- To simplify debugging by isolating faults

Finding defects early during unit testing reduces cost and complexity in later testing stages.

3 Scope of Unit Testing

Due to its limited scope, unit testing focuses on:

- Internal processing logic
- Local data structures
- Control flow paths
- Boundary conditions
- Error-handling mechanisms

The relative complexity of tests is constrained because only one unit is examined at a time.

4 Unit Test Considerations

Effective unit testing requires careful attention to the following aspects.

4.1 Module Interface Testing

Module interface testing ensures that data correctly flows into and out of the component under test.

If data does not enter or exit a component correctly, all other unit tests are meaningless.

This is always the **first** step in unit testing.

4.2 Testing Local Data Structures

Local data structures are examined to ensure:

- Data integrity is preserved during execution
- Temporary data is correctly stored and modified
- No unintended side effects occur

If possible, the impact of local data on global data structures should also be observed.

4.3 Independent Path Testing

Independent path testing ensures that every unique execution path through the control structure is exercised at least once.

Test cases are designed to detect:

- Incorrect computations
- Faulty comparisons
- Improper branching or looping

4.4 Boundary Condition Testing

Boundary testing evaluates a module's behavior at the limits of allowable input and data ranges.

Common boundary failures occur when:

- The last element of an array is accessed
- A loop executes its first or last iteration
- Maximum or minimum input values are used

Test cases should include values:

just below, at, and just above boundaries

4.5 Error-Handling Path Testing

Error-handling code must be tested explicitly.

There is a strong tendency to write error-handling code and never test it.

Potential error-handling defects include:

- Incorrect or unclear error messages
- Mismatch between reported and actual error
- System crash before error handling executes
- Incorrect exception processing
- Insufficient diagnostic information

5 Antibugging and Error Anticipation

A good software design anticipates errors and defines clear error-handling paths.

Antibugging is the practice of designing software to handle errors gracefully rather than assuming errors will never occur.

Failure to test error-handling paths can result in catastrophic failures during real usage.

6 Unit Test Procedures

Unit testing is commonly performed as an adjunct to the coding phase.

6.1 Test Case Design

- Unit tests may be designed **before** or **after** coding.
- Design documents guide test case creation.
- Each test case must include expected results.

Designing unit tests before coding helps produce cleaner, more testable code.

7 Unit Test Environment

Because a component is not a standalone program, additional software is often required.

7.1 Driver

A **driver** is a program that invokes the component under test, supplies test data, and displays results.

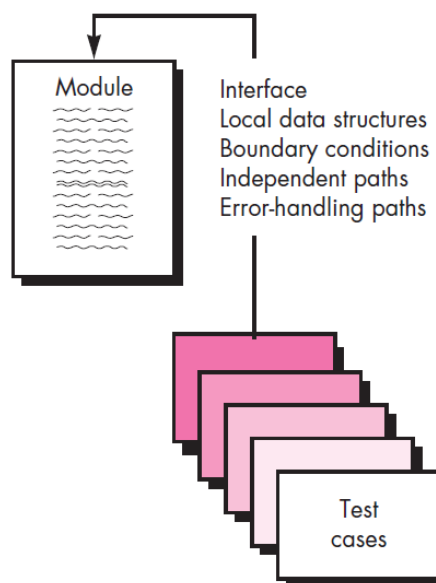
7.2 Stub

A **stub** is a dummy module that simulates subordinate components called by the unit under test.

Stubs:

- Use the subordinate module's interface
- Perform minimal processing
- Return control to the calling unit

Drivers and stubs represent testing overhead and are not part of the final software product.



8 Impact of Design on Unit Testing

Unit testing is simplified when components exhibit **high cohesion**.

- Single-purpose components require fewer test cases
- Errors are easier to predict and isolate

- Debugging becomes more efficient

Good design directly improves testability.

9 Summary

- Unit testing verifies individual software components.
- Focuses on logic, data structures, and control flow.
- Boundary and error-handling tests are critical.
- Drivers and stubs support isolated testing.
- High cohesion leads to more effective unit testing.